



Manipulação de imagens em Python para apoiar o ensino de Pensamento Computacional **no Ensino Médio**



Oficina

Pensamento
Computacional

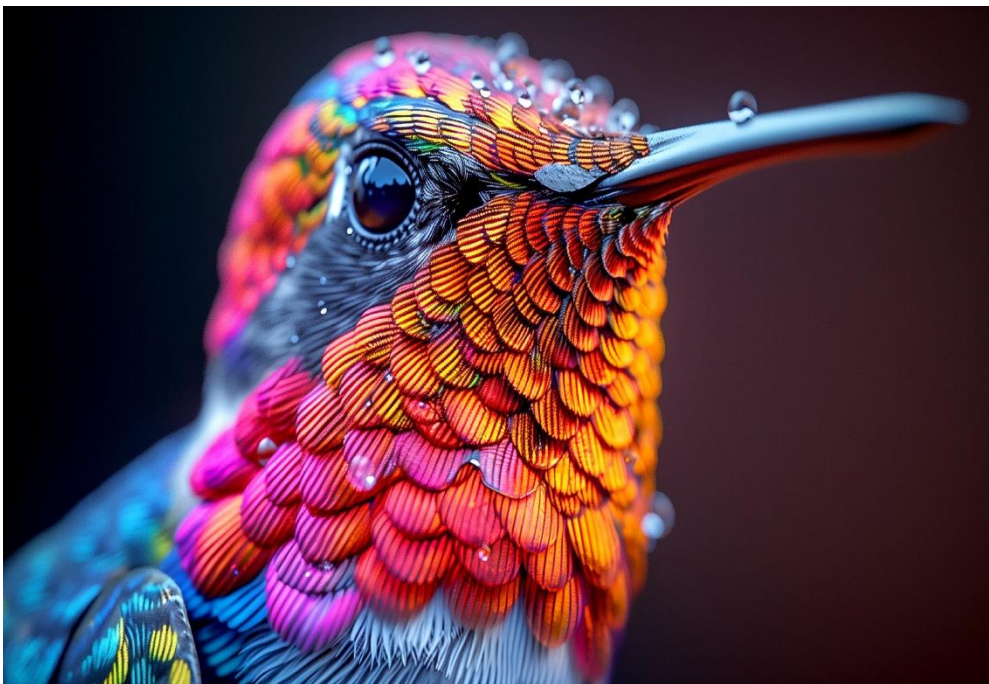


Figura 1. Imagem adaptada com uso de IA.

IMAGE M

O Que é Uma *Imagem Digital?*

Uma imagem é uma **função bidimensional $f(x, y)$** onde a intensidade de luz é mapeada em coordenadas espaciais precisas.

Para o computador, o mundo real analógico torna-se uma **grade organizada de números** e dados numéricos, permitindo o processamento digital.

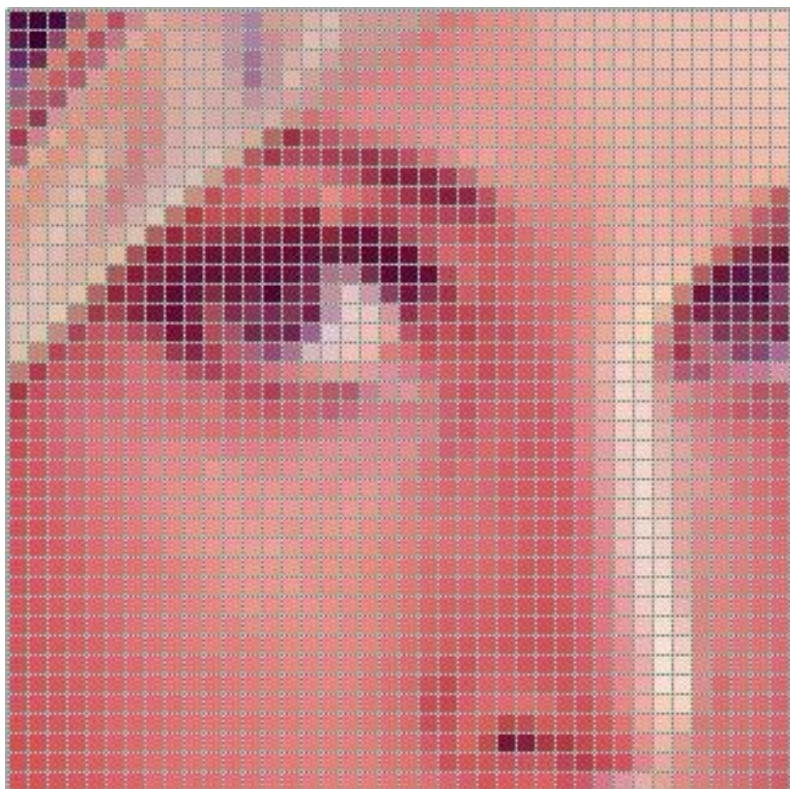


Figura 2. Pixels ampliados em imagem.



Estrutura Digital

Uma imagem digital é uma grade de pixels, funcionando como uma matriz matemática de números inteiros que armazena cor.

A Representação Matricial



Grade de Pixels

A estrutura fundamental que forma as imagens digitais modernas.



Dados Computáveis

Associação entre imagens e matrizes de números inteiros para processamento.



Visualização de Dados

Cada célula armazena valores de cor, transformando visão em informação.

CONCEITO BASE

O Coração da Imagem: O Pixel



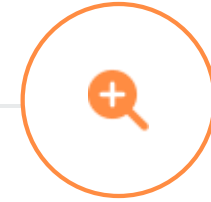
Unidade Mínima

O Pixel (Picture Element) é a menor unidade componente de uma imagem digital.



Posição e Cor

A combinação exata de coordenadas e valores cromáticos define a representação visual final.



Zoom e Contorno

Ao ampliar, observamos pequenos quadrados que, juntos, formam os contornos da percepção.

Valores de *Intensidade e Tons*

Cada pixel possui um valor numérico que define sua luminosidade. Alterar esses números muda a tonalidade, traduzindo luz em dados matemáticos.

8 Bits

PADRÃO DE COR

0-255

ESCALA DE TONS



CONCEITO BASE

Representação da Luz

A escala de valores numéricos é utilizada para representar com precisão luz e sombra em cada ponto da imagem digital.



PADRÃO BINÁRIO

Escala de Intensidade

O padrão de 0 (preto absoluto) a 255 (branco puro) define a profundidade em imagens comuns de 8 bits por canal.



MANIPULAÇÃO

Alteração de Tonalidade

Como pequenas mudanças nos coeficientes numéricos alteram a percepção visual e a tonalidade final da composição.

Resolução Espacial e DPI

A resolução define a clareza pela densidade de pontos. O **DPI (Dots Per Inch)** indica a quantidade de pontos por polegada; quanto maior o valor, mais detalhes são percebidos.

Imagens de alta resolução exibem contornos nítidos, enquanto a baixa resolução resulta em fotos "quadriculadas" ou sem nitidez.



Figura 4 – Diferença entre DPI.

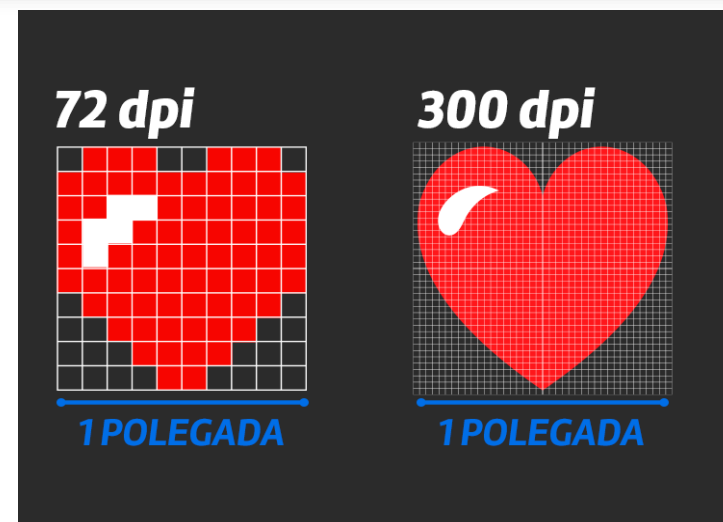


Figura 5 – Diferença entre DPI.

Imagens Binárias: Preto ou Branco

O tipo mais simples de imagem digital, operando com apenas dois valores possíveis por pixel para representar a informação visual.

1Bit

POR PIXEL

0 / 1

VALORES BINÁRIOS



Figura 7 – Imagem Binária.

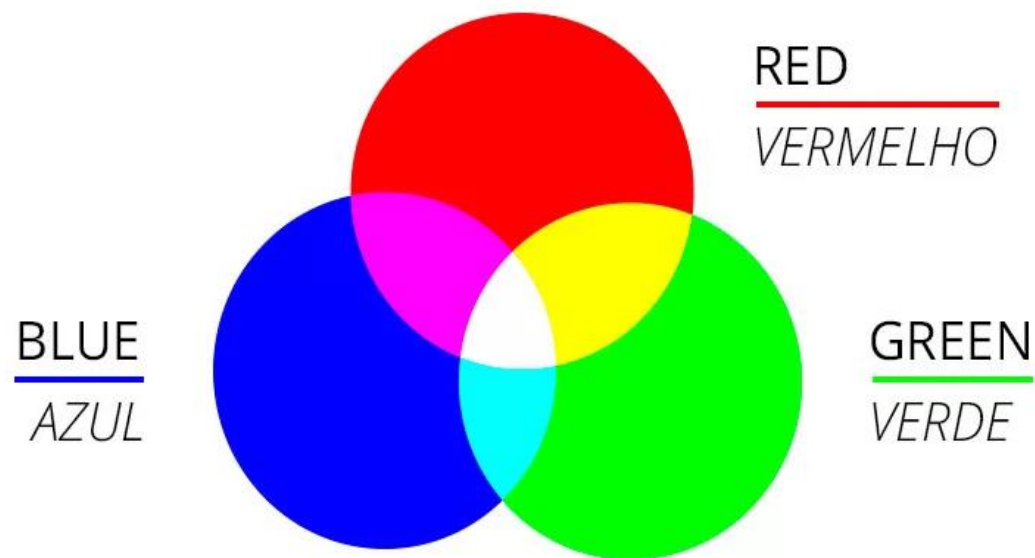


Figura 8 – Representação do modelo de cores RGB.

COLOURS

O Modelo de *Cores RGB*

As cores são formadas pela mistura aditiva de Red, Green e Blue. Cada canal possui 8 bits, totalizando 24 bits por pixel.

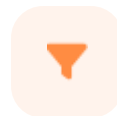
Variando a intensidade de 0 a 255 em cada componente, criamos mais de 16 milhões de cores: o branco é a soma máxima e o preto é a ausência de luz.

Filtragem de *Imagens*



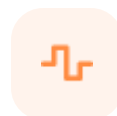
Visão Computacional

Filtrar imagens remove ruídos e destaca detalhes essenciais, preparando os dados para uma análise automatizada mais precisa e eficiente.



O que é um Filtro?

Uma lente aplicada para processar e melhorar a informação visual bruta.



Espaço e Frequência

Processamento em pixels (espaço) ou variação de brilho (frequência).



Ferramenta de Melhoria

Essencial para destacar informações e reduzir interferências indesejadas.

PROCESSAMENTO

Filtragem no Domínio do Espaço



Operação Direta

Atuamos diretamente sobre os pixels individuais da imagem original.



Vizinhança

Cada pixel é transformado sob a influência direta dos seus vizinhos.



Aplicações

Ideal para o realce de detalhes e redução de ruídos ou imperfeições.

TÉCNICA DE SUAVIZAÇÃO

Suavização:

Filtro de Média

Reduz o ruído calculando a média aritmética dos pixels vizinhos para substituir o valor central.

Gera um efeito de "blur" ideal para uniformizar texturas e suavizar imperfeições da imagem.

Suavização: *Filtro da Mediana*



OBJETIVO PRINCIPAL

Técnica ideal para remover ruído "sal e pimenta" sem borrar a imagem original.

Valor Central vs. Média

Ao selecionar o valor central dos pixels vizinhos, o filtro elimina valores extremos (pontos pretos e brancos) de forma cirúrgica, superando a média aritmética.

EFICÁCIA ESTATÍSTICA

Preservação de Detalhes

Diferente de outros filtros de suavização, a mediana mantém os contornos nítidos e detalhes estruturais, evitando o efeito indesejado de embaçamento.

RESULTADO DE ALTA FIDELIDADE ✓

Filtro Gaussiano

O uso da curva de Gauss para atribuir pesos aos pixels, criando uma suavização que imita a visão humana.

Blur

SUAVIZAÇÃO NATURAL

Focus

EFEITO DE CÂMERA

Visão Computacional

Essencial no pré-processamento para reduzir o ruído e preparar a imagem para algoritmos de detecção.

TÉCNICA

Pesos Pixels

APLICAÇÃO

Pré-processo



IMAGE M

Segmentação de *Imagens*

A arte de **separar objetos do fundo**, focando em regiões de interesse. É o passo essencial para o reconhecimento de objetos.

- ✓ Isolar uma flor específica em um jardim vasto.
- ✓ Destacar um rosto em uma foto de grupo.

Binarização e *Limiarização*

Processamento

Utiliza um limiar (threshold) para separar pixels claros de escuros, transformando a imagem em preto e branco de forma eficiente.



Separação de Tons

Uso de limiar para separar pixels claros e escuros.



Máscaras Binárias

Criação de máscaras a partir de tons de cinza.

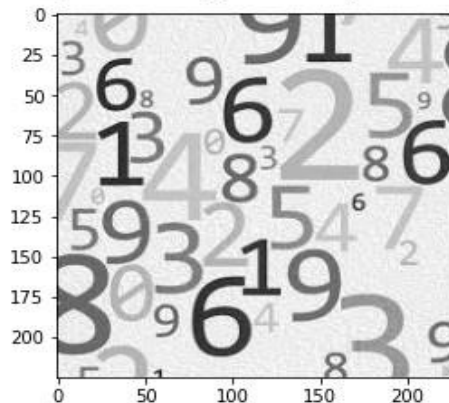


Binarização Inversa

Invertendo os papéis de objeto e fundo na imagem.

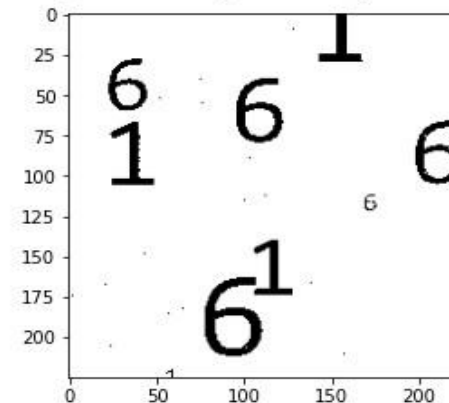
```
▶ img=cv2.imread("/content/drive/My Drive/Colab Notebooks/imagen2.jpg",cv2.IMREAD_GRAYSCALE)  
pyplot.imshow(img,cmap='gray')
```

<matplotlib.image.AxesImage at 0x7f4d9555c198>



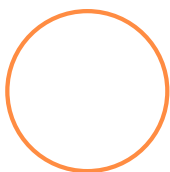
```
[68] limiar , imglimiar = cv2.threshold(img,80,255,cv2.THRESH_BINARY)  
pyplot.imshow(imglimiar, cmap='gray')
```

<matplotlib.image.AxesImage at 0x7f4d9542fb38>



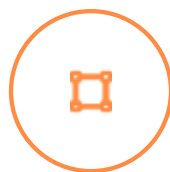


Extração de Características



Análise de Pixels

Transformação de pixels brutos em informações úteis e descritivas para o sistema.



Vetores de Dados

Utilização de vetores de características para representar objetos de forma matemática.



Simplificação

Redução de dimensionalidade essencial para simplificar dados e otimizar o processamento.

Descritores

- . É uma análise muito importante pois diversos objetos podem ser caracterizados por sua forma.
- . A partir da forma, vários atributos podem ser calculados, tais como área, perímetro, raio, dentre outros.
- . Geralmente, esses descritores são extraídos a partir da imagem binarizada.

```
[75] imgf=cv2.imread("/content/drive/My Drive/Colab Notebooks/imagem3.png",cv2.IMREAD_GRAYSCALE)
```

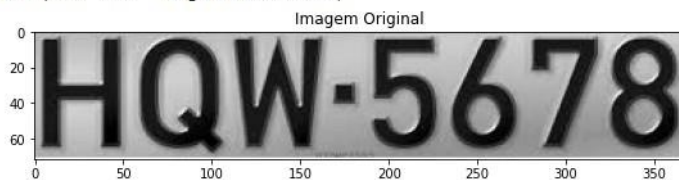
```
[76] limiar , imglimiari = cv2.threshold(imgf,70,255,cv2.THRESH_BINARY_INV)
```

```
fig = pyplot.figure(figsize=(20,20))
```

```
ax1 = fig.add_subplot(121)  
pyplot.imshow(imgf, cmap='gray')  
pyplot.title("Imagem Original")
```

```
ax2 = fig.add_subplot(122)  
pyplot.imshow(imglimiari, cmap='gray')  
pyplot.title("Imagem Binarizada")
```

```
Text(0.5, 1.0, 'Imagem Binarizada')
```



```
moments = cv2.moments(imglimiari)  
huMoments = cv2.HuMoments(moments)  
moments
```

```
{ 'm00': 2168265.0,  
  'm01': 76074150.0,  
  'm02': 3338521710.0,  
  'm03': 163518194100.0,  
  'm10': 394148400.0,  
  'm11': 13678307610.0,  
  'm12': 600712465800.0,  
  'm20': 97586494170.0,  
  'm21': 3354585237630.0,  
  'm30': 27366246113550.0,  
  'mu02': 669440071.8134775,  
  'mu03': -589684883.0094604,  
  'mu11': -150493068.81924057,  
  'mu12': 4394337813.276215,  
  'mu20': 25937982008.17476,  
  'mu21': -14549250493.139404,  
  'mu30': 196878552750.66797,  
  'nu02': 0.00014239245771783453,  
  'nu03': -8.518028634841009e-08,  
  'nu11': -3.201044998788412e-05,  
  'nu12': 6.347643699736984e-07,  
  'nu20': 0.0055171077470457155,  
  'nu21': -2.1016467589189985e-06,  
  'nu30': 2.8439208774652297e-05}
```

VISÃO COMPUTACIONAL

Os Invariantes *de Hu*



CONCEITO CHAVE

Uma genialidade matemática que cria uma assinatura digital única para objetos, garantindo o reconhecimento preciso.

Os 7 Momentos

Introdução aos 7 Momentos de Hu e sua genialidade matemática no reconhecimento independente de rotação ou escala de objetos complexos.

PROPRIEDADE DE INVARIÂNCIA

Assinatura Digital

Mesmo que uma imagem seja girada ou aumentada, ela mantém a mesma identidade digital através da estabilidade dos cálculos matemáticos de Hu.



PROCESSAMENTO DIGITAL

Transformação

Logarítmica de Hu

Os Momentos de Hu variam em escalas gigantescas. Aplicamos o logaritmo para estabilizar os dados e torná-los comparáveis em sistemas complexos.

Escala

VARIABILIDADE REDUZIDA

IA

PRECISÃO AUMENTADA



Vetores de IA

Esta técnica prepara os vetores para algoritmos de Inteligência Artificial com maior precisão, otimizando o reconhecimento de padrões.

[VER DOCUMENTAÇÃO](#) →

OBJETIVO

Estabilidade de Dados

FOCO

Escalabilidade

Redes Neurais na *Classificação*

Bibliotecas

```
[3] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Importar os dados

```
[17] data=pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Iris.csv')
iris=pd.DataFrame(data)
iris.head()
```

Informação do repositório

```
[5] iris.info()
```

Gráficos

```
[6] plt.figure(figsize=(8,4))
sns.countplot(x='Species', data=iris);
```

```
[7] sns.set_style('whitegrid')
sns.FacetGrid(iris, hue = 'Species', height = 6).map(plt.scatter, 'SepalLengthCm', 'SepalWidthCm').add_legend()
plt.show()
```

```
[8] sns.set_style('whitegrid')
sns.FacetGrid(iris, hue = 'Species', height = 6).map(plt.scatter, 'PetalLengthCm', 'PetalWidthCm').add_legend()
plt.show()
```



Cérebro Digital

As redes neurais funcionam como um cérebro digital que aprende a distinguir padrões complexos nos dados de forma automática.



Aprendizado de Padrões

Como o computador aprende a distinguir padrões nos dados.



Treinamento e Momentos de Hu

Uso dos 7 Momentos de Hu como entrada para descrever formas.



Resultados e Precisão

Alta precisão na tarefa de identificação e classificação de imagens.

Redes Neurais na *Classificação*

```
[9] from sklearn.neural_network import MLPClassifier
```

```
[10] #Separando dados de entrada (X) e dados de saída (Y)
iris=iris.values
X = iris[:, 1:5].astype(float)
Y = iris[:, 5].astype(object)
print(X.shape)
print(Y.shape)
```

```
[11] # Separa os dados em treinamento e teste
from sklearn import model_selection
X_train, X_test, Y_train, Y_test = model_selection.train_test_split(X, Y, test_size=0.3, random_state=0)
print(X_train.shape)
print(Y_train.shape)
```

```
[12] # Modelo de classificação
model=MLPClassifier(activation='relu',hidden_layer_sizes=(3),random_state=0,max_iter=20000,verbose=True)
```

```
[13] # Roda o modelo com os dados de treinamento
model.fit(X_train,Y_train)
```

```
[14] # Predição
prediction=model.predict(X_test)
```

▶ Y_test

```
▶ from sklearn import metrics
metrics.accuracy_score(prediction, Y_test)
```



Referências

- MAGNIFIC. *Home page*. Disponível em: <https://www.magnific.com/br>.
- KYRIACOU, Jack. Pixels. WordPress, 18 dez. 2007. Disponível em: jackkyriacou.wordpress.com/2007/12/18/pixels/.
- JORDÃO, Fabio. *Entenda quais são as diferenças entre o PPI e o DPI*. [TecMundo](#). Publicado em: 18 ago. 2014.
- EQUIPE EDITORIAL. Como ampliar uma imagem sem perder a qualidade? ArteRef. 21 dez. 2020. Disponível em: <https://arteref.com/fotografia/como-ampliar-uma-imagem-sem-perder-a-qualidade/>.
- SIRAKOV, Nikolay M. Image Processing with Applications – Meeting 2. Texas A&M University-Commerce, 2013. Disponível em: https://faculty.tamuc.edu/nsirakov/Teaching/lecturepowerpoints/imageprocessing/Meeting2_IPwA.pdf.
- UNIVERSIDADE ESTADUAL DE CAMPINAS (UNICAMP). Tarefa 07 – Imagens. Disponível em: <https://www.ic.unicamp.br/~lehilton/cursos/1s2021/mc102w/tarefas/07-imagens/>.
- AFIXGRAF. O que significa RGB? Disponível em: <https://www.afixgraf.com.br/blog/o-que-significa-rgb/>.
- DATACAMP. Machine Learning in R for beginners. Disponível em: <https://www.datacamp.com/tutorial/machine-learning-in-r>.